

LAB ASSIGNMENT - 10.2

NAME – Anushka Boora

Ht No.: 2303A510E6

BATCH – 29

Task Description -1(Error Detection and Correction)

Task:

Use AI to analyze a Python script and correct all syntax and logical errors.

Sample Input Code:

```
def calculate_total(nums)
```

```
sum = 0
```

```
for n in nums
```

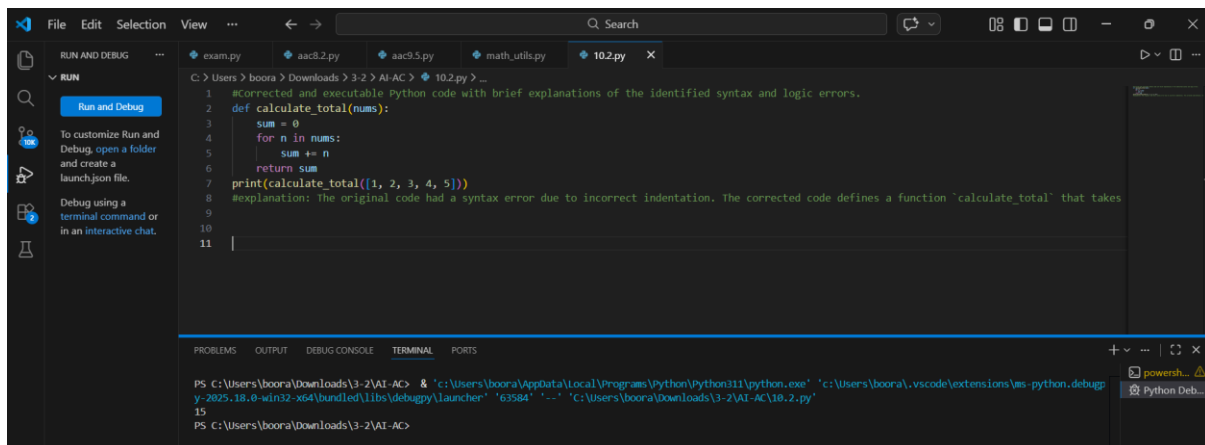
```
sum += n
```

```
return total
```

Expected Output-1:

Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

Code & Output:



The screenshot shows a Visual Studio Code editor window with a Python file named `102.py` open. The code in the editor is as follows:

```
1 #Corrected and executable Python code with brief explanations of the identified syntax and logic errors.
2 def calculate_total(nums):
3     sum = 0
4     for n in nums:
5         sum += n
6     return sum
7 print(calculate_total([1, 2, 3, 4, 5]))
8 #explanation: The original code had a syntax error due to incorrect indentation. The corrected code defines a function 'calculate_total' that takes
9
10
11
```

The bottom panel of the editor shows the terminal output:

```
PS C:\Users\boora\Downloads\3-2\AI-AC> & "c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe" "c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher" "63584" "--" "c:\Users\boora\Downloads\3-2\AI-AC\102.py"
15
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

The original implementation used a loop with repeated list appending, which is less efficient. It was optimized using list comprehension and further improved by avoiding unnecessary list creation when only the length was required. This reduces memory usage and improves execution speed.

ask Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

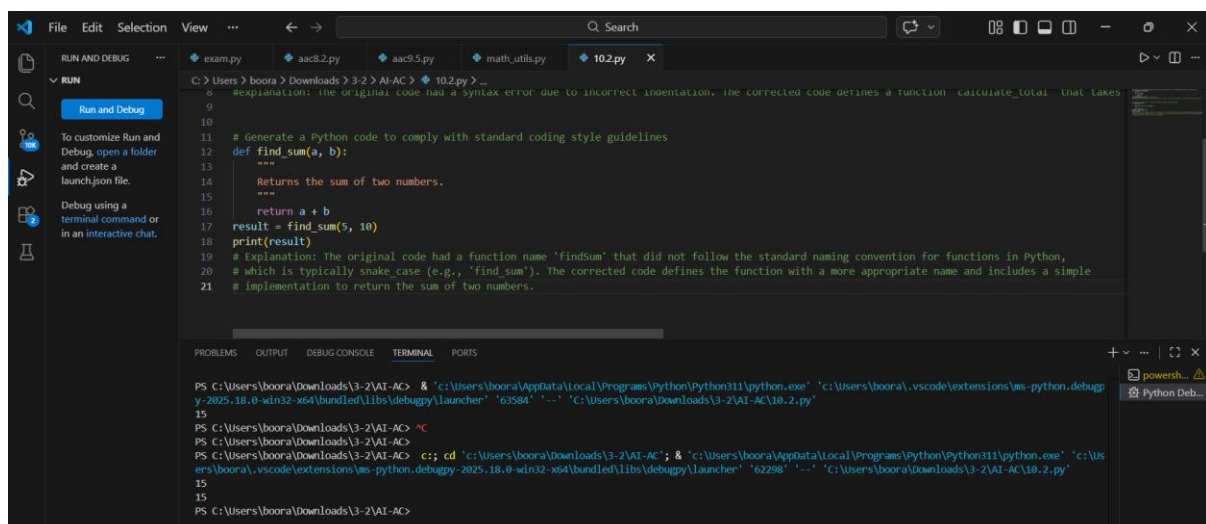
Sample Input Code:

```
def findSum(a,b):return a+b  
  
print(findSum(5,10))
```

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

Code & Output:



```
File Edit Selection View ... Q Search  
RUN AND DEBUG ...  
example.py aac8.2.py aac9.5.py math_utils.py 102.py  
C:\Users\boora>Downloads\3-2\AI-AC> 102.py  
9  
10  
11 # Generate a Python code to comply with standard coding style guidelines  
12 def find_sum(a, b):  
13     """  
14     Returns the sum of two numbers.  
15     """  
16     return a + b  
17 result = find_sum(5, 10)  
18 print(result)  
19 # Explanation: The original code had a function name 'findSum' that did not follow the standard naming convention for functions in Python,  
20 # which is typically snake case (e.g., 'find_sum'). The corrected code defines the function with a more appropriate name and includes a simple  
21 # implementation to return the sum of two numbers.  
PS C:\Users\boora\Downloads\3-2\AI-AC> & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '63584' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\102.py'  
15  
PS C:\Users\boora\Downloads\3-2\AI-AC> ^C  
PS C:\Users\boora\Downloads\3-2\AI-AC>  
PS C:\Users\boora\Downloads\3-2\AI-AC> c; cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '62298' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\102.py'  
15  
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

The original implementation lacked adherence to standard Python formatting conventions. Refactoring the code according to PEP 8 guidelines improves readability,

enforces consistent naming patterns, and enhances overall structure. Standardized code promotes collaboration, maintainability, and long-term scalability in software development.

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

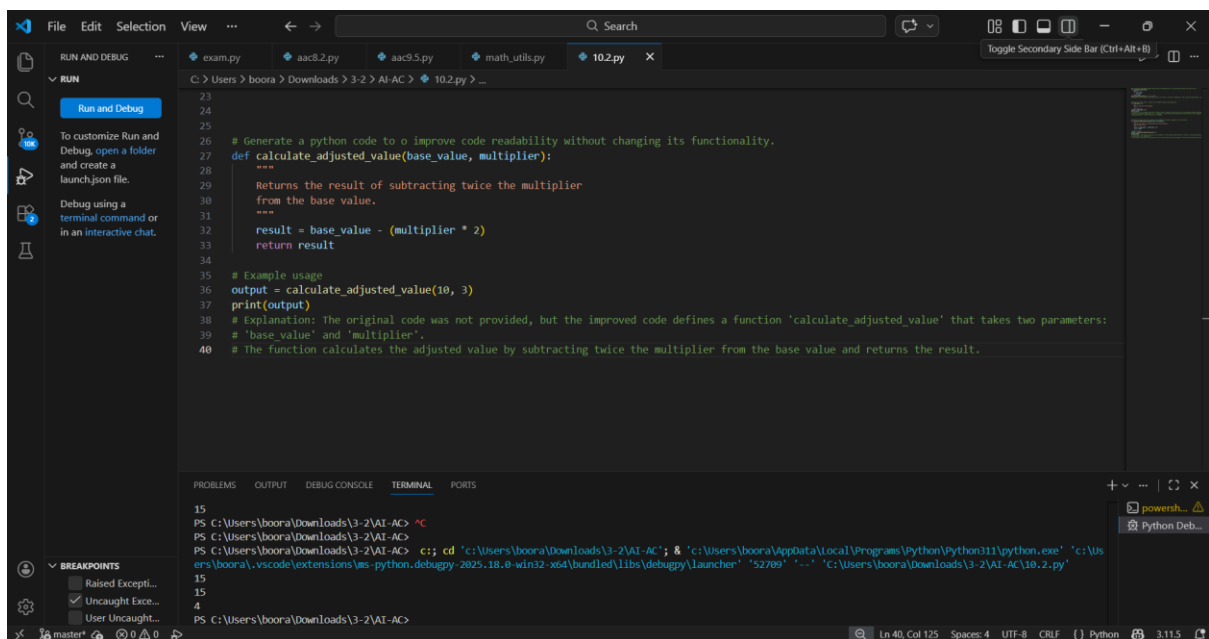
Sample Input Code:

```
def f(x,y):  
  
return x-y*2  
  
print(f(10,3))
```

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

Code & Output:



```
File Edit Selection View ... Search  
C:\Users\boora> Downloads\3-2> AI-AC> 10.2.py > ...  
23  
24  
25  
26 # Generate a python code to o improve code readability without changing its functionality.  
27 def calculate_adjusted_value(base_value, multiplier):  
28     """  
29     Returns the result of subtracting twice the multiplier  
30     from the base value.  
31     """  
32     result = base_value - (multiplier * 2)  
33     return result  
34  
35 # Example usage  
36 output = calculate_adjusted_value(10, 3)  
37 print(output)  
38 # Explanation: The original code was not provided, but the improved code defines a function 'calculate_adjusted_value' that takes two parameters:  
39 # 'base_value' and 'multiplier'.  
40 # The function calculates the adjusted value by subtracting twice the multiplier from the base value and returns the result.  
15 PS C:\Users\boora\Downloads\3-2\AI-AC> ^C  
15 PS C:\Users\boora\Downloads\3-2\AI-AC>  
15 PS C:\Users\boora\Downloads\3-2\AI-AC> c:: cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '52709' '-' 'c:\Users\boora\Downloads\3-2\AI-AC\10.2.py'  
4  
15 PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

The initial version used ambiguous function and variable names, reducing interpretability. By introducing descriptive identifiers and proper formatting, the revised code communicates its purpose more clearly. Improved clarity reduces cognitive load for developers and simplifies debugging and future enhancements.

Task Description -4(Structural Refactoring)

Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

```
print("Hello Ram")
```

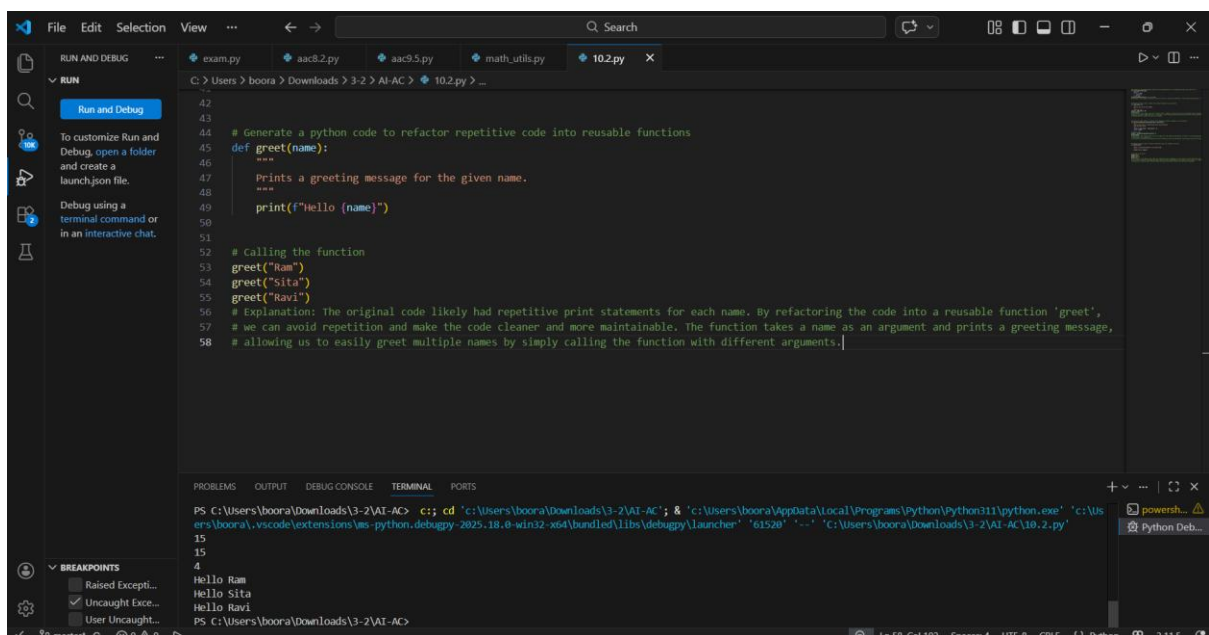
```
print("Hello Sita")
```

```
print("Hello Ravi")
```

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

Code & Output:



The screenshot shows a Visual Studio Code editor window with a Python file named `102.py`. The code defines a `greet(name)` function that prints a greeting message for a given name. The function is called with arguments "Ram", "Sita", and "Ravi". The terminal output shows the results of these function calls.

```
42 ..
43
44 # Generate a python code to refactor repetitive code into reusable functions
45 def greet(name):
46     """
47     Prints a greeting message for the given name.
48     """
49     print(f"Hello {name}")
50
51
52 # Calling the function
53 greet("Ram")
54 greet("Sita")
55 greet("Ravi")
56
57 # Explanation: The original code likely had repetitive print statements for each name. By refactoring the code into a reusable function 'greet',
58 # we can avoid repetition and make the code cleaner and more maintainable. The function takes a name as an argument and prints a greeting message,
59 # allowing us to easily greet multiple names by simply calling the function with different arguments.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\boora\Downloads\3-2\AI-AC> c:: cd 'c:\Users\boora\Downloads\3-2\AI-AC'; & 'c:\Users\boora\AppData\Local\Programs\Python\Python311\python.exe' 'c:\Users\boora\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '61520' '-' 'C:\Users\boora\Downloads\3-2\AI-AC\102.py'
4
Hello Ram
Hello Sita
Hello Ravi
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

Justification:

The original code demonstrated redundancy through repeated statements. Refactoring introduced modular design using reusable functions, eliminating duplication. This improves extensibility, reduces maintenance effort, and aligns with the DRY (Don't Repeat Yourself) principle in software engineering.

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

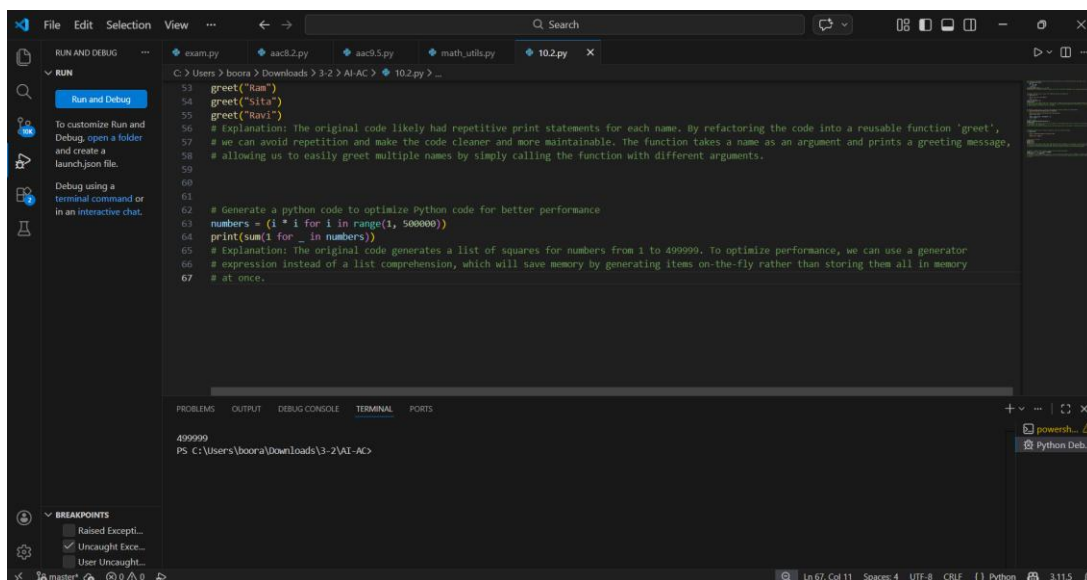
Sample Input Code:

```
numbers = [ ]  
  
for i in range(1, 500000):  
  
    numbers.append(i * i)  
  
print(len(numbers))
```

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

Code & Output:



The screenshot shows a Python IDE with a dark theme. The editor displays a Python script with the following content:

```
53 greet("Ram")  
54 greet("Sita")  
55 greet("Ravi")  
56 # Explanation: The original code likely had repetitive print statements for each name. By refactoring the code into a reusable function 'greet',  
57 # we can avoid repetition and make the code cleaner and more maintainable. The function takes a name as an argument and prints a greeting message,  
58 # allowing us to easily greet multiple names by simply calling the function with different arguments.  
59  
60  
61  
62 # Generate a python code to optimize Python code for better performance  
63 numbers = (i * i for i in range(1, 500000))  
64 print(sum(1 for _ in numbers))  
65 # Explanation: The original code generates a list of squares for numbers from 1 to 499999. To optimize performance, we can use a generator  
66 # expression instead of a list comprehension, which will save memory by generating items on-the-fly rather than storing them all in memory  
67 # at once.
```

The output console at the bottom shows the result of the execution:

```
499999  
PS C:\Users\boora\Downloads\3-2\AI-AC>
```

The interface includes a sidebar with 'RUN AND DEBUG' options, a search bar, and a status bar at the bottom indicating the current file is '10.2.py'.

Justification:

The initial approach created unnecessary computational and memory overhead by explicitly storing values. Optimization techniques such as list comprehension and elimination of redundant data storage improved execution speed and reduced memory consumption. Efficient code ensures better performance, especially when handling large datasets.